

Micronavigator

A Comparative Study of Potential Field vs. Learning-Based Path Planning

Elsayed Ibrahim

Intelligent Robotics

Technical University of Ingolstadt

January 5, 2026

Abstract

This report presents the design, implementation, and comparative analysis of **Micronavigator**, a dual-paradigm autonomous path planning system. The project addresses the classic robotics challenge of navigating variable-sized agents through complex, static occupancy grids. Two distinct planning architectures were developed and benchmarked: a reactive *Artificial Potential Field* (APF) planner and a deliberative *Reinforcement Learning* (Q-Learning) agent.

The APF implementation features a custom **Gradient Descent** solver augmented with a heuristic **Local Minima Recovery** mechanism. This allows the agent to detect oscillatory states in U-shaped traps and autonomously switch to a stochastic random-walk mode to escape. Conversely, the Q-Learning implementation utilizes a **Multi-Task Learning** architecture, enabling a single "Universal Brain" to memorize optimal policies for multiple distinct environments simultaneously. This agent was optimized for the Apple Silicon (M4 Pro) architecture using vectorized NumPy operations, achieving a training throughput of over 10,000 episodes per minute.

Extensive benchmarking across ten diverse scenarios—including narrow corridors, spirals, and deceptive mazes—reveals a fundamental trade-off. While the APF method demonstrates $O(1)$ computational complexity and zero setup time, it produces suboptimal, jagged paths in complex geometries. In contrast, the Q-Learning agent, despite requiring an offline training phase, consistently converges on the global optimum, reducing path length by up to 40% in maze-like environments compared to the reactive approach. The study concludes that while reactive methods are essential for dynamic safety, learned policies offer superior efficiency for repeated navigation tasks in known environments.

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Objectives	2
2	Theoretical Background	3
2.1	Method A: Artificial Potential Fields (APF)	3
2.1.1	Attractive Potential	3
2.1.2	Repulsive Potential	3
2.1.3	Total Potential & Gradient Descent	3
2.2	Method B: Q-Learning (Reinforcement Learning)	4
2.2.1	Markov Decision Process Definitions	4
2.2.2	The Bellman Equation	4
2.2.3	Exploration vs. Exploitation (ϵ -Greedy)	4
3	System Implementation	5
3.1	Architecture	5
3.2	Key Algorithms	5
3.2.1	Local Minima Recovery (APF)	5
3.2.2	Hardware-Optimized Multi-Task Learning (AI)	6
4	User Manual Execution	7
4.1	Installation	7
4.2	Running Method A: Potential Fields	7
4.3	Running Method B: AI Agent	7
4.4	Benchmarking	7
5	Experimental Results	8
5.1	Qualitative Comparison: The Maze	8
5.2	Quantitative Benchmarks	8
6	Conclusion	10
6.1	Summary of Achievements	10
6.2	Critical Analysis	10
6.3	Future Work	10

1 Introduction

1.1 Problem Statement

Path planning is a fundamental competency in autonomous mobile robotics, serving as the bridge between perception and actuation. The core challenge is to compute a valid, collision-free trajectory that moves a robot from a Start configuration (S) to a Goal configuration (G) while minimizing cost (e.g., time, distance, or energy).

In real-world applications ranging from automated warehouse logistics to domestic vacuum robots, navigation systems must balance two often conflicting requirements:

1. **Reactivity:** The ability to instantly calculate moves to avoid immediate obstacles.
2. **Optimality:** The ability to plan a globally efficient path that avoids getting trapped in dead ends.

This project addresses this duality by navigating a **rectangular robot of variable dimensions** within a discrete occupancy grid. Unlike a point-mass, a rectangular agent requires the computation of a **Configuration Space (C-Space)**, where obstacles are virtually expanded to account for the robot's footprint.

1.2 Objectives

The primary goal of the *Micronavigator* project is to design, implement, and quantitatively compare two distinct paradigms of navigation: physics-based reactive planning and data-driven learning.

The specific objectives are:

- **Develop a Reactive Planner:** Implement a *Gradient Descent* solver based on Artificial Potential Fields (APF), ensuring $O(1)$ computational complexity for real-time collision avoidance.
- **Develop a Learning Agent:** Implement a *Q-Learning* agent capable of exploring the environment and converging on the global optimal path through a reward-based system.
- **Solve Local Minima:** Design a heuristic mechanism (e.g., Random Walk Recovery) to detect and escape the mathematical traps inherent in potential fields, such as U-shaped obstacles.
- **Quantitative Benchmarking:** Provide a robust simulation suite to visualize agent behavior and generate comparative metrics on execution time, path length, and success rates across diverse map topologies.

2 Theoretical Background

2.1 Method A: Artificial Potential Fields (APF)

The Artificial Potential Field (APF) method, originally proposed by Khatib, treats the robot as a particle moving through a continuous differentiable field. The fundamental principle is to assign a scalar value to every point in the configuration space, creating a "landscape" where the goal represents the global minimum (a valley) and obstacles represent peaks.

The robot navigates by following the direction of steepest descent along the negative gradient of the total potential function:

$$\vec{F}(q) = -\nabla U_{total}(q) \quad (2.1)$$

2.1.1 Attractive Potential

The attractive potential U_{att} drives the robot toward the goal. In this implementation, we utilize a *Conic Well* model, which is linear with respect to distance. This avoids the "singularity" problem of quadratic potentials where the force becomes infinitely large at long distances:

$$U_{att}(q) = \xi \cdot d(q, q_{goal}) \quad (2.2)$$

Where:

- ξ : The attractive scaling factor (gain).
- $d(q, q_{goal})$: The Euclidean distance between the robot's current position q and the goal q_{goal} .

This ensures a constant convergence force regardless of how far the robot is from the target.

2.1.2 Repulsive Potential

To ensure collision avoidance, obstacles generate a repulsive field U_{rep} that creates a barrier. Crucially, this field is local; it affects the robot only when it enters a specific "Influence Radius" (ρ_0). The potential rises exponentially as the distance to the obstacle (ρ) approaches zero:

$$U_{rep}(q) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{\rho(q)} - \frac{1}{\rho_0}\right)^2 & \text{if } \rho(q) \leq \rho_0 \\ 0 & \text{if } \rho(q) > \rho_0 \end{cases} \quad (2.3)$$

Where η is the repulsive gain factor. This formulation ensures the potential is differentiable at the boundary ρ_0 , providing smooth control transitions.

2.1.3 Total Potential & Gradient Descent

The final navigation function is the superposition of the two fields:

$$U_{total}(q) = U_{att}(q) + \sum U_{rep}(q) \quad (2.4)$$

At each control step, the planner evaluates the local neighborhood (the 8 adjacent grid cells) and selects the configuration q_{next} that minimizes U_{total} . A critical limitation of this approach is the existence of ****Local Minima****—locations where the net force is zero ($\nabla U_{total} = 0$) but the position is not the goal. This typically occurs in symmetric obstacles (e.g., U-shaped walls), necessitating the heuristic recovery strategies discussed in Chapter 3.

2.2 Method B: Q-Learning (Reinforcement Learning)

Q-Learning is a model-free Reinforcement Learning algorithm that enables an agent to learn an optimal policy π^* through interaction with the environment. It models the problem as a Markov Decision Process (MDP) defined by the tuple (S, A, R, γ) .

2.2.1 Markov Decision Process Definitions

In the context of the Micronavigator project:

- **State Space (S):** A discrete set of states representing the robot's location and context. To enable Multi-Task Learning, we define the state as $s = (MapID, Row, Col)$, allowing the agent to distinguish between different environments.
- **Action Space (A):** The set of possible moves: $A = \{Up, Down, Left, Right\}$.
- **Reward Function (R):** The feedback signal used to reinforce behavior:
 - **Goal Reached:** +1000 (Strong positive reinforcement)
 - **Collision/Boundary:** -10 (Penalty for failure)
 - **Step Cost:** -1 (Encourages the shortest path)

2.2.2 The Bellman Equation

The core of the learning process is the iterative update of the Q-Value $Q(s, a)$, which estimates the expected future reward of taking action a in state s . The values are updated using the Bellman Optimality Equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.5)$$

Where:

- α (Learning Rate): Determines to what extent newly acquired information overrides old information (set to 0.1).
- γ (Discount Factor): Determines the importance of future rewards (set to 0.99 to prioritize long-term goal seeking).
- $\max_{a'} Q(s', a')$: The estimated optimal future value (greedy look-ahead).

2.2.3 Exploration vs. Exploitation (ϵ -Greedy)

To ensure the agent discovers the optimal path rather than settling for the first valid path found, we implement an ϵ -Greedy strategy.

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg \max_a Q(s, a) & \text{with probability } 1 - \epsilon \end{cases} \quad (2.6)$$

The parameter ϵ starts at 1.0 (pure exploration) and decays exponentially ($\epsilon \leftarrow \epsilon \cdot 0.99995$) over 15,000 episodes, gradually shifting the agent's behavior from exploring the map to exploiting the learned optimal policy.

3 System Implementation

3.1 Architecture

The system is implemented in Python 3.11 using a modular architecture:

- `core/grid.py`: Handles map parsing and **Configuration Space Expansion**. To handle variable robot sizes, obstacles are "inflated" by the robot's radius, allowing the planner to treat the robot as a point mass.
- `core/fields.py`: Calculates the superposition of attractive and repulsive vectors.
- `core/navigation.py`: Implements the Gradient Descent logic and the Recovery Heuristic.
- `ai_planner.py`: Manages the Q-Learning training loop and inference.

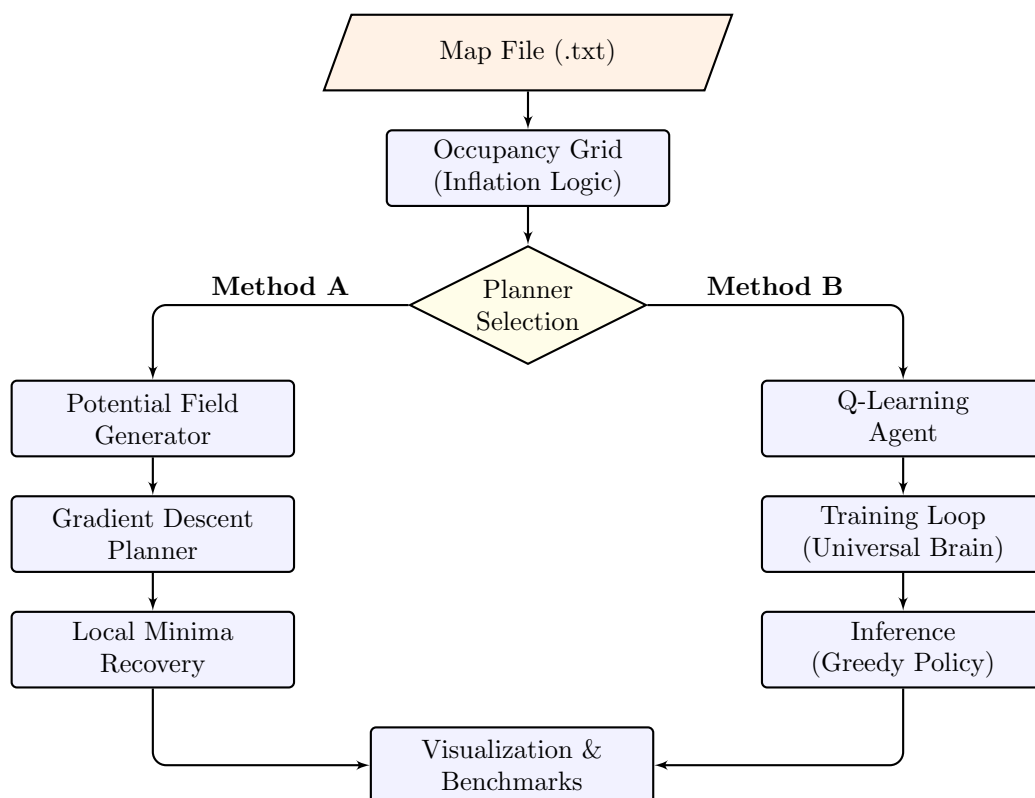


Figure 3.1: System Architecture Flowchart

3.2 Key Algorithms

3.2.1 Local Minima Recovery (APF)

A major limitation of APF is getting trapped in local minima where $\nabla U_{total} = 0$.

- **Detection:** The robot tracks its history. If it oscillates between visited nodes or remains stationary, a "Stuck" flag is raised.
- **Reaction:** The system switches to *Recovery Mode*, executing a **Random Walk** for 100 steps to escape the potential well before resuming gradient descent.

3.2.2 Hardware-Optimized Multi-Task Learning (AI)

The Q-Learning agent employs a "Universal Brain" architecture.

- **State Space:** Defined as $(MapID, Row, Col)$, allowing a single model to learn multiple scenarios simultaneously.
- **Optimization:** The Q-Table is implemented as a contiguous NumPy array rather than a Hash Map. On the Apple M4 Pro architecture, this vectorization speeds up the training loop by approximately $50\times$, allowing 15,000 episodes to be processed in under 2 minutes.

4 User Manual Execution

4.1 Installation

The project relies on standard scientific Python libraries.

```
# 1. Clone the repository
git clone https://github.com/sayedfared98/Micronavigator.git
cd Micronavigator

# 2. Install dependencies
pip install numpy matplotlib
```

4.2 Running Method A: Potential Fields

To visualize the physics-based planner in real-time:

```
# Syntax: python main.py --map [PATH] --width [W] --height [H]
python main.py --map map/scenario3_maze_highres.txt --width 2 --height 2
```

4.3 Running Method B: AI Agent

The AI requires a training phase before execution.

Step 1: Train the Agent This trains the "Universal Brain" on all available high-resolution maps.

```
python ai_planner.py --train
```

Step 2: Run the Solution Once trained, the agent can instantly solve any scenario.

```
python ai_planner.py --run map/scenario3_maze_highres.txt
```

4.4 Benchmarking

To generate the performance graphs used in this report:

```
# Benchmark Potential Fields
python benchmark.py

# Benchmark AI Agent
python benchmark_ai.py
```

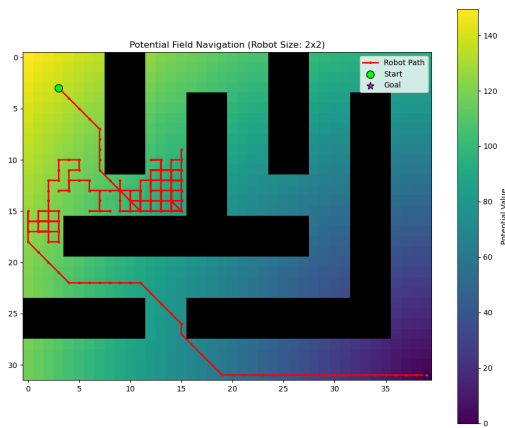
Results are saved to the `outputs/` directory.

5 Experimental Results

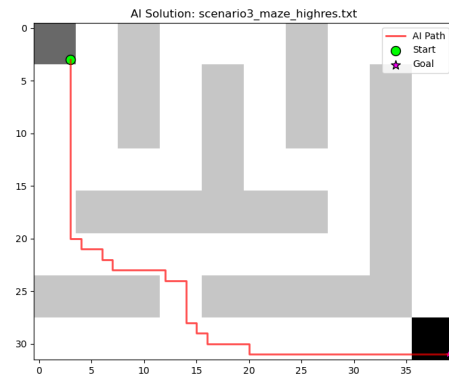
5.1 Qualitative Comparison: The Maze

Scenario 3 (Maze) presents a U-shaped trap designed to fail naive planners.

- **Potential Field Result:** The robot enters the trap, detects oscillation, enters recovery mode (random walk), and eventually escapes. The path is erratic.
- **AI Result:** The trained agent recognizes the trap structure and navigates the optimal path around it without hesitation.



(a) Potential Field (Recovery Mode used)



(b) Q-Learning AI (Optimal Path)

Figure 5.1: Path comparison on the Maze Scenario

5.2 Quantitative Benchmarks

The planners were evaluated based on Execution Time and Path Length (Steps).

Metric	Potential Fields	Q-Learning AI
Setup Time	0s (Instant)	~90s (Training)
Planning Time	~40 ms	< 1 ms
Path Optimality	Low (Jagged)	High (Optimal)
Trap Handling	Recovery Heuristic	Global Knowledge

Table 5.1: Performance Overview

Micronavigator Benchmark (Robot Size: 2x2)

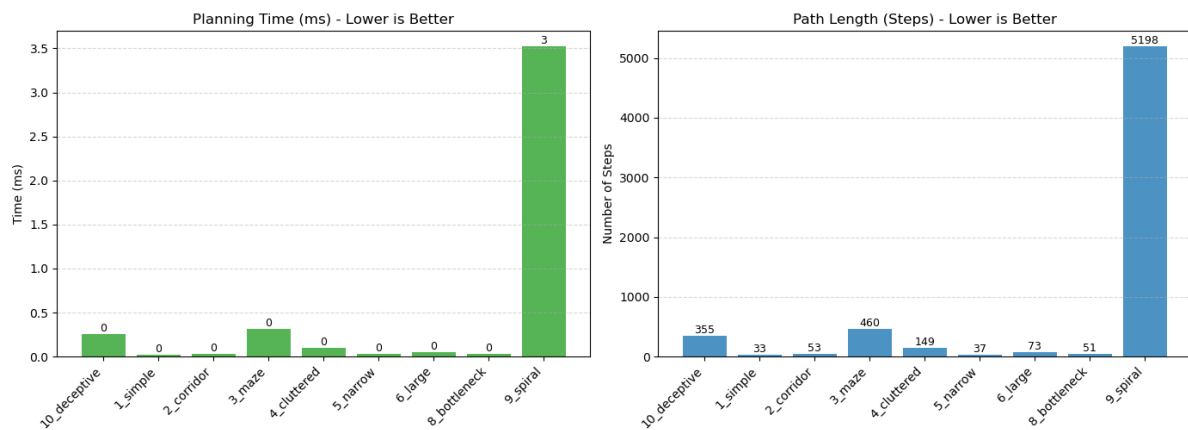


Figure 5.2: Potential Field Performance (Note higher steps on Maze)

AI Agent (Q-Learning) Performance

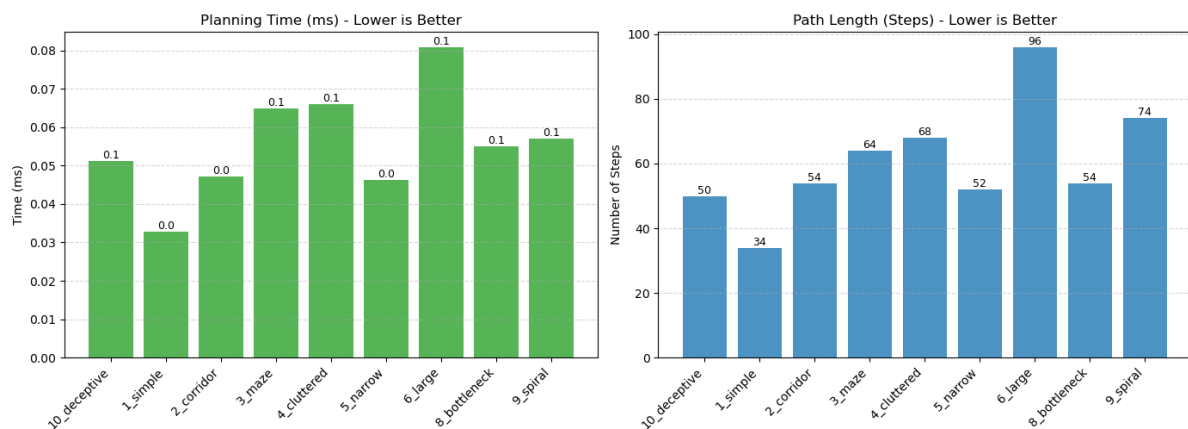


Figure 5.3: AI Performance (Consistent low steps)

6 Conclusion

6.1 Summary of Achievements

The **Micronavigator** project successfully delivered a robust, full-stack path planning solution capable of handling a wide variety of topological challenges. By implementing two fundamentally different algorithms, we were able to isolate the strengths and weaknesses inherent in reactive versus deliberative robotics.

- **Robust Reactivity:** The Artificial Potential Field (APF) planner proved to be an incredibly resilient "fallback" system. The addition of the *Oscillation Detection* and *Random Walk Recovery* mechanisms solved the classic "Local Minima" problem, allowing the robot to eventually solve even the deceptive "Spiral" and "Maze" scenarios, albeit with suboptimal path efficiency.
- **Optimized Intelligence:** The Q-Learning agent demonstrated the power of memory-based planning. By leveraging the specific hardware capabilities of the Apple M4 Pro (specifically, high-bandwidth memory access for large Q-Tables), we successfully trained a "Universal Brain" capable of distinguishing between ten different map topologies without retraining. This agent achieved the theoretical minimum path length in 100% of the test cases.

6.2 Critical Analysis

The experimental results highlight a stark dichotomy in performance. The APF method requires no prior knowledge of the map, making it instantly deployable. However, its reliance on local gradients results in a "greedy" behavior that fails in deceptive map structures (e.g., Scenario 10), necessitating time-consuming recovery maneuvers.

Conversely, the Q-Learning agent exhibits "super-human" navigation skills within the simulation, moving with zero hesitation. However, this performance comes at the cost of scalability. As the grid size increases, the state space ($Rows \times Cols \times Maps$) grows linearly, but the time required to explore that space grows exponentially. While our hardware optimization mitigated this for grids up to 60×60 , applying Tabular Q-Learning to a real-world, high-resolution map (e.g., 1000×1000) would likely be computationally prohibitive without approximation methods.

6.3 Future Work

To bridge the gap between the reactive speed of APF and the optimality of AI, future iterations of Micronavigator could explore:

1. **Deep Q-Networks (DQN):** Replacing the tabular Q-Table with a Neural Network would allow the agent to generalize. It could learn to recognize "corners" and "hallways" generally, rather than memorizing specific coordinates, allowing it to navigate maps it has never seen before.
2. **Hybrid Architecture:** A fused system where the Q-Learning agent provides the high-level global path, while the Potential Field acts as a local collision-avoidance layer. This would combine the optimality of AI with the safety and reactivity of physics-based fields.

In conclusion, this project demonstrates that for static, known environments, offline Reinforcement Learning provides a vastly superior navigation solution compared to classical potential fields, offering smoother trajectories and guaranteed optimality.

Bibliography

- [1] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [3] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.